TECHNICAL UNIVERSITY OF MUNICH · DEPARTMENT OF INFORMATICS

RT-LLM-Proxy: A Real-Time WebRTC-to-LLM Bridge with Adaptive Audio Processing

Zeyu Li
Department of Informatics
Technical University of Munich
Munich, Bavaria, Germany
lizeyu1101@gmail.com

ABSTRACT

We present RT-LLM-Proxy, a production-grade Go proxy that bridges browser clients communicating over WebRTC to streaming large language model (LLM) providers via WebSocket. The system terminates Opus-encoded audio, transcodes to a canonical 48 kHz mono signed-16 PCM format, and forwards to streaming LLM voice provider WebSocket APIs. We identify and address three key engineering challenges: (i) monotonically growing latency from per-frame timing accumulation, resolved with a session-level wall-clock ticker; (ii) GC pressure from per-frame heap allocations eliminated via buffer reuse achieving 0 allocs/op; and (iii) CPU saturation under high concurrency, mitigated by an adaptive Opus complexity controller. Benchmarks on a 16-core host yield ~107 full-duplex sessions per CPU core, with zero-allocation buffer reuse removing ~500,000 heap allocations per second at 5,000 sessions. The adaptive complexity controller (*sessions* mode) reduces pacing SLO violations from 21.2% to 15.6% under 150-session load while remaining stable, outperforming a reactive controller that oscillates under sustained load. Together these optimisations make a thin, single-host, provider-agnostic bridge a practical foundation for production real-time voice interaction with large language models.

1 Introduction

The emergence of multimodal large language models capable of real-time voice dialogue creates demand for low-latency audio bridging infrastructure. Modern browsers communicate using WebRTC [1], which transports Opus-encoded [2] audio as RTP [3] packets over a peer connection. Streaming LLM voice providers expose WebSocket APIs with provider-specific audio formats and binary framing protocols that browsers cannot reach directly. A dedicated proxy layer is therefore required to terminate the WebRTC peer connection, decode Opus audio to PCM, and forward to the appropriate provider while pacing the return stream in real time.

Building such a proxy exposes a class of engineering challenges specific to the real-time audio domain. Unlike typical web services where latency jitter is tolerable, a voice LLM proxy must maintain steady 20 ms frame intervals [5] in both directions: falling behind produces audible artefacts and growing end-to-end latency; getting ahead overruns the browser's jitter buffer. Furthermore, audio encode/decode operations run on the hot path for every concurrent session, making per-frame memory allocation a first-class concern at scale — one that interacts adversarially with Go's concurrent garbage collector.

This work makes the following contributions:

(i) We design and implement RT-LLM-Proxy, a production-grade Go proxy supporting multiple streaming LLM voice providers through a provider-agnostic model interface, with production features including

atomic Redis rate limiting, per-provider circuit breaking, session reconnect replay, and a Kafka-backed Protobuf transcript side-channel.

(ii) We analyse and resolve three fundamental real-time audio engineering problems — clock-drift-free outbound pacing, zero-allocation codec buffers, and adaptive CPU load shedding via Opus complexity — documenting the precise failure modes that motivate each decision.

(iii) We benchmark the proxy on a 16-core host, establishing a practical capacity model of ~107 sessions/core, identifying the SLO knee at ~50 co-located sessions, and demonstrating 26% improvement in pacing violations with adaptive complexity.

[11] R. Jesup, S. Loreto, and M. Tüxen. *WebRTC Data Channels.* RFC 8831, IETF, January 2021.

Section 2 provides background. Section 3 describes the proxy architecture. Section 4 details engineering optimisations. Section 5 evaluates the system. Sections 6–7 discuss limitations and conclude.

2 Background

2.1 WebRTC and the Opus Codec

WebRTC [1] is a browser-native real-time communication standard using RTP over UDP for media transport. Opus [2] is the mandatory audio codec, operating at 6–510 kbps with 2.5–60 ms frame durations. The default WebRTC configuration uses 20 ms frames at 48 kHz. A key parameter for this work is Opus [2] *complexity* (0–10, as defined in RFC 6716), which trades perceptual quality for encode CPU — the lever at the core of our adaptive load-shedding design (Section 4.3).

2.2 Streaming LLM Audio APIs

Streaming LLM voice providers expose persistent WebSocket APIs that accept PCM audio and return synthesised speech. Provider APIs are heterogeneous: uplink sample rates vary (16 kHz is common), downlink formats differ (s16le, f32le, gzip-framed binary), and some carry per-chunk MIME-type rate metadata while others use protocol-fixed constants. None are reachable directly from browsers due to CORS restrictions and binary framing complexity; a dedicated proxy layer is required.

2.3 Positioning in the Proxy Design Space

Agent orchestration frameworks such as Pipecat [10] and LiveKit Agents [9] provide session routing, TURN relay, and horizontal scale at the cost of framework lock-in. RT-LLM-Proxy occupies a complementary point: a thin, single-host, provider-agnostic bridge that makes real-time audio engineering explicit and observable, with seam-based extensibility rather than embedded policy.

3 System Architecture

3.1 End-to-End Data Flow

The proxy is a thin, stateful bridge: it terminates the browser's WebRTC [1] peer connection, transcodes Opus [2] ↔ PCM, and forwards to the LLM provider via the Model seam, pacing the synthesised reply back to the browser. Transcript text is additionally exchanged over the WebRTC data channel [11] as monotonically sequenced JSON events, providing the reconnect protocol with a reliable sequence anchor. Figure 1 summarises the architecture. A loopback fake provider is included for load testing, allowing benchmarking without upstream API cost or latency.

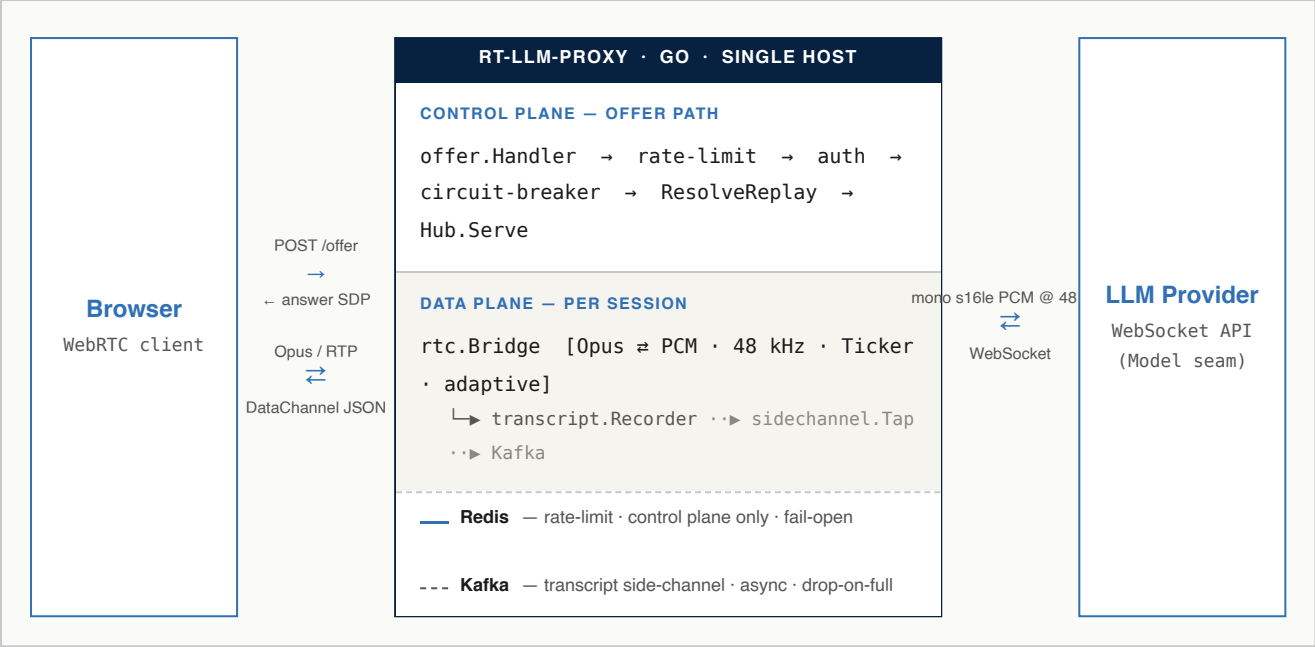

Figure 1. RT-LLM-Proxy system architecture. The *control plane* handles SDP offer negotiation, rate limiting (Redis, fail-open), authentication, per-provider circuit breaking, and session replay resolution — all on the offer path, never in the media loop. The *data plane* runs per session: `rtc.Bridge` terminates the WebRTC [1] peer connection, transcodes Opus [2] ↔ PCM, and paces outbound frames via a session-level `Ticker`; `transcript.Recorder` assigns monotonic sequence numbers shared by the data channel and the side-channel; and `sidechannel.Tap` publishes events to Kafka asynchronously (dashed path, best-effort, drop-on-full). Solid arrows are on the hot path; dashed are optional.

3.2 Audio Contract

All audio crossing the Model seam is **mono s16le PCM at 48 kHz** [3] [2]. Providers convert to/from their own wire formats internally, keeping the bridge fully provider-agnostic. This invariant is load-bearing: adding a new provider requires implementing a single four-method interface (`SendAudio`, `SendText`, `Recv`, `Close`) without touching the bridge. One concrete adapter reads the sample rate per-chunk from MIME metadata (the safer pattern); another uses a protocol-fixed const whose binary framing cannot carry a rate field. These asymmetries are intentional — flattening the dynamic case to a static const would silently discard a safety property.

Table 1. Per-provider audio format conversions. All rates are upsampled or downsampled to/from the 48 kHz contract inside the adapter; the Bridge never knows the wire rate.

PROVIDER	UPLINK WIRE RATE	DOWNLINK WIRE FORMAT	RATE SOURCE
Provider A	16 kHz s16le	s16le (variable)	Per-chunk MIME type
Provider B	16 kHz s16le	f32le 24 kHz (gzip)	Fixed const
Loopback	(discarded)	s16le 48 kHz sine	—

3.3 Module Structure

Table 2. Key Go packages and their responsibilities. Arrows show the only allowed dependency directions; the Bridge never imports a provider package.

PACKAGE	ROLE
`internal/rtc`	Terminates WebRTC peer connection; paces outbound audio; owns `Recorder`
`internal/model`	Provider-agnostic `Model` interface seam; optional `Transcriber`
`internal/model/<provider>`	Concrete provider adapters; each implements the `Model` interface with its own WebSocket protocol, audio format, and rate negotiation
`internal/audio`	libopus encode/decode (cgo); linear resampler
`internal/offer`	SDP HTTP handler; rate-limit check; circuit-breaker; replay resolution
`internal/ratelimit`	Redis fixed-window limiter; atomic Lua INCR+EXPIRE; fail-open
`internal/modelcb`	Per-provider circuit breaker; transient vs auth error classes
`internal/sidechannel`	Kafka Protobuf publisher; async bounded channel; drop-on-full
`internal/transcript`	Session-scoped `Recorder`; monotonic seq authority
`internal/adaptive`	Opus complexity controllers: *sessions* and *drift*
`cmd/loadgen`	pion headless clients; pre-encoded Opus replay; polls `/stats`

3.4 Production Features

Rate Limiting.

The SDP offer endpoint is rate-limited using a Redis fixed-window counter per client IP. The `INCR` and `EXPIRE` operations are executed atomically via a single Lua script — a separate two-step sequence risks leaving a key without a TTL on mid-operation crash, permanently locking out that IP. The limiter *fails open*: a Redis blip allows the request (logging the error) rather than dropping the call.

Circuit Breaker.

Provider WebSocket connections are wrapped in per-provider circuit breakers [7]. After a configurable number of consecutive transient failures (default 5), the breaker opens for 30 s, returning `503 + Retry-After`. Authentication failures (401/403) open with a longer hold (5 min default). Breakers are per-provider with optional per-provider overrides; an outage on one provider does not affect sessions on other providers.

Session Reconnect Replay.

Each session's `Recorder` assigns monotonic sequence numbers to transcript lines. On reconnect, the client sends `X-Session-ID` and `X-Last-Seq` headers; the proxy replays missed lines from memory or from a Kafka [6] tail scan (`--replay-kafka`), bounded by a configurable timeout (300 ms default) to keep reconnect latency predictable. Replay is provider-scoped: sessions on one provider cannot replay into a connection on a different provider type.

Kafka Transcript Side-Channel.

A `Tap` implements `transcript.Listener` and publishes `TranscriptEvent` Protobufs to Kafka [6], partitioned by `user_id` (falling back to `session_id` for anonymous users to avoid hot-partition skew). The pipeline is async, bounded-channel, drop-on-full — it degrades the transcript feature gracefully but never blocks the media path.

4 Engineering Optimisations

4.1 Real-Time Outbound Pacing Without Clock Drift

Problem.

The naive implementation of outbound audio pacing uses `time.Sleep(frameDur)` or `time.After(frameDur)` between frames. Under this scheme each frame's real period is *frameDur + encode_time*, so cumulative lag grows linearly with response length. For a 10-second response at 50 fps and encode time ~161 μs, lag accumulates to ~80 ms — audible temporal smear. The failure is not intermittent; it is deterministic and proportional to response length.

Solution.

A single `time.Ticker` is initialised at session start and fires every 20 ms on a fixed wall-clock schedule. The `writeOutbound` goroutine blocks on the ticker channel; encode work fits *inside* the 20 ms window rather than adding to it. The ticker channel's buffer of 1 coalesces missed ticks during model silence, so resuming speech after a pause does not burst a backlog of frames. We empirically validated this design by reverting to a shared slot-sharded timing wheel — it worsened pacing dramatically (n=50: 2.6% → 19%; n=150: 28% → 76% frames ≥30 ms) because the wheel advances on received ticks rather than wall time, and CPU contention under load caused the pacer goroutine to miss ticks, dragging the entire fleet.

4.2 Zero-Allocation Audio Hot Path (P6)

Problem.

The baseline `Encoder` and `Decoder` each allocated one buffer per frame: 1.25 KiB (Encode), 6 KiB (Decode), 7.25 KiB (roundtrip). At 50 frames/s per full-duplex session, 5,000 sessions generate ~500,000 heap allocations per second. Go's GC must scan and evacuate this short-lived memory continuously, producing minor-GC pauses that manifest as pacing jitter.

Solution.

Both `Encoder` and `Decoder` now hold an internal reuse buffer (bufio.Scanner style). The returned slice is valid only until the next call — a *borrowed* lifetime. This is safe because: (a) provider adapters resample audio into a newly allocated output slice before forwarding downstream; and (b) pion's Opus payloader copies the payload in `WriteSample` before returning. The result is 0 allocs/op for all three operations (Figure 2). Decode latency improves by 4.6%; encode latency is unchanged, confirming that the encode wall is compute, not allocation overhead.

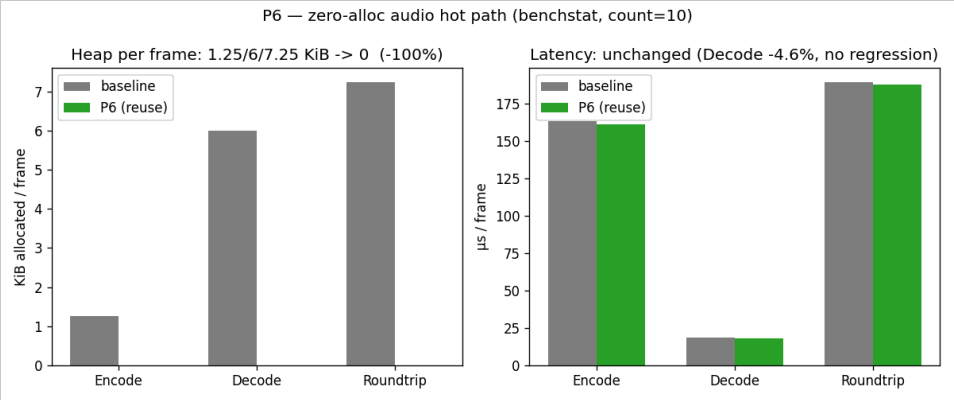

Figure 2. P6 zero-alloc optimisation (benchstat, count=10). *Left*: heap allocated per frame. Encode 1.25 KiB → 0, Decode 6 KiB → 0, Roundtrip 7.25 KiB → 0 (–100%). *Right*: latency per frame. Encode is unchanged (compute-bound); Decode improves 4.6% due to reduced GC pressure. At 5,000 sessions this removes ~500,000 allocs/s.

4.3 Adaptive Opus Complexity

Complexity as the Encode-CPU Lever.

Opus [2] encode dominates decode 8.5× at default complexity 10 (~161 μs/frame). Reducing complexity is the only operation that moves the encode wall. Table 3 shows measured encode latency and derived session capacity at each level. Complexity 5 halves encode CPU (~79 μs, 2.1× capacity) and is typically transparent for 16 kbps narrowband speech; complexity 0 produces audible loss.

Table 3. Opus encode latency and derived sessions-per-core as a function of complexity. Measurements on a 16-core host, 48 kHz, 20 ms frames, rich audio signal. Complexity 8→10 is essentially free.

COMPLEXITY	ENCODE / FRAME	VS C=10	SESSIONS / CORE	NOTES
10 (default)	~166 μs	1.0×	~107	Maximum quality
8	~170 μs	~same	~107	8→10 is free; skip
5	~79 μs	**2.1×**	**~200**	**Recommended default**
3	~60 μs	2.8×	~270	Adaptive floor
0	~35 μs	4.7×	~330	Audible quality loss

Controller Design.

Complexity is a live-adjustable atomic read per frame. Two controller strategies are provided via `--adaptive`:

Sessions mode (A) is a proactive step function of concurrent session count with hysteresis: idle → c=10, >40 sessions → c=5, >90 sessions → c=3. It reduces quality before the SLO breaks, is deterministic, and settles stably.

Drift mode (B) is reactive on the ≥30 ms pacing histogram bucket. Under sustained load it exhibits a feedback oscillation: lowers complexity until pacing recovers, then raises it, causing pacing to break again (Figure 3). This is the same feedback-loop hazard as the reverted timing wheel. Mode B is retained as a labelled experimental option; Mode A is the recommended default.

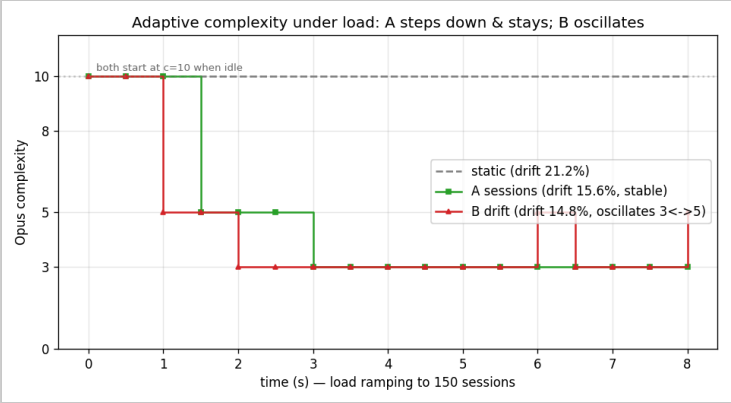

Figure 3. Adaptive complexity under a 150-session ramp. Static (c=10 throughout): 21.2% cumulative frames ≥30 ms. Mode A (*sessions*): 15.6%, steps down at t≈1 s then t≈2 s and holds stable. Mode B (*drift*): 14.8% but oscillates between c=3 and c=5 continuously, exhibiting the feedback-loop hazard described in text. Mode A is preferred.

4.4 Atomic Rate Limiting and Fail-Open

A naïve two-step `INCR` + `EXPIRE` sequence has a crash window: if the process dies between the two Redis commands the key retains no TTL and that IP is permanently locked out. The implementation executes both as a single Lua script, making the pair atomic at the Redis level. The limiter fails open [7] on Redis error: rate limiting is a soft guard; a Redis blip must not take down the real-time media path.

4.5 Opus SDP Negotiation for Lossy Links

Opus is tuned independently for each direction of the session via SDP `fmtp` attributes [4], trading bandwidth and fidelity for resilience.

Uplink (browser → proxy). The proxy's SDP answer advertises:

```
minptime=10; useinbandfec=1; usedtx=1; maxaveragebitrate=16000
```

`useinbandfec=1` enables in-band FEC so later packets can recover earlier loss; `usedtx=1` activates discontinuous transmission, suppressing silent frames to reduce jitter-buffer pressure; `minptime=10` allows 10 ms packetisation for shorter first-packet latency on short utterances; and `maxaveragebitrate=16000` caps the uplink at 16 kbps — sufficient for narrowband voice dialogue with LLMs.

Downlink (proxy → browser). The encoder is initialised with `AppVoIP` application mode [2], in-band FEC [4], DTX, and `PacketLossPerc=10` — the encoder-side flag that actually activates FEC generation (the fmtp field alone is insufficient). Frames are 20 ms / 960 samples at 48 kHz, paced by the session-level `Ticker` (§4.1).

5 Evaluation

5.1 Experimental Setup

All rt-llm-proxy benchmarks run on a 16-core host. The load generator (`cmd/loadgen`) drives pion [8] WebRTC [1] sessions with pre-encoded Opus [2] audio replayed at real time against the loopback provider to isolate proxy CPU from upstream LLM latency. Unless noted, loadgen and proxy run on the same host (pessimistic; co-located loadgen steals CPU from the proxy). The Opus micro-benchmark uses Go's `testing.B` with `count=10`; statistical comparison uses `benchstat`.

5.2 Opus Micro-Benchmark

~161 µs	~18 µs	~107	0
ENCODE / FRAME (C=10)	DECODE / FRAME	SESSIONS / CORE (OPUS ONLY)	ALLOCS/OP AFTER P6

Table 4 shows baseline and P6 results. Opus [2] encode dominates decode 8.5× at ~161 µs/frame. At 50 frames/s per full-duplex session, one core can sustain ~107 sessions for Opus alone (0.95% of a core per session). The computed 16-core ceiling is ~600–1000 sessions after deducting headroom for SRTP/DTLS, pion goroutine scheduling, and GC. After P6 buffer reuse all allocation counts reach zero with no encode latency regression; decode improves 4.6%, attributable to reduced minor GC pause frequency.

Table 4. Opus micro-benchmark before and after P6 zero-alloc optimisation (benchstat, count=10, 16-core host). Encode CPU is unchanged (compute-bound); Decode improves 4.6%; all allocation counts reach zero.

OPERATION	ALLOCS/OP (BEFORE)	ALLOCS/OP (AFTER)	B/OP (BEFORE)	B/OP (AFTER)	LATENCY Δ
Encode	1	0	1,280 B	0	—
Decode	1	0	6,144 B	0	–4.6%
Roundtrip	2	0	7,424 B	0	—

5.3 Capacity Sweep

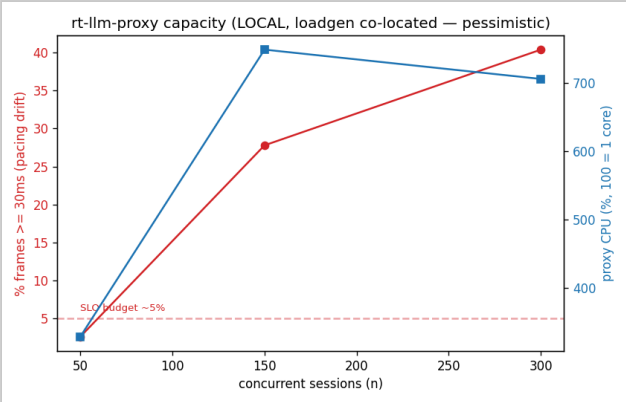

Figure 4. RT-LLM-Proxy capacity under co-located load testing (loadgen and proxy on the same 16-core host, loopback provider). Red: % frames ≥30 ms (pacing drift, SLO budget ~5% dashed). Blue: proxy CPU (100 = 1 core). The SLO is healthy at n=50 (2.6%) and broken at n=150 (27.8%). These numbers are pessimistic — co-located loadgen steals ~3–7 cores from the proxy at n=50–150 (WebRTC [1] sessions).

Table 5 summarises the capacity sweep. The pacing SLO (~5% frames ≥30 ms) is healthy at n=50 and broken at n=150, with further degradation at n=300. The SLO breakpoint is artificially low due to CPU co-location: the loadgen's pion clients steal CPU from the proxy on the same machine. Off-box loadgen measurements would maintain healthy pacing to the ~600–1000 ceiling predicted by the micro-benchmark. At n=300 pion spawns ~17,800 proxy-side goroutines, introducing a secondary scheduling and memory pressure limit independent of Opus CPU.

Table 5. Capacity sweep summary (co-located loadgen, loopback provider). All-connected counts reflect sessions that completed SDP handshake.

N	PROXY CPU (CORES)	FRAMES ≥30 MS	CONNECTED	STATUS
50	~3.3	2.6%	50/50	✓ SLO healthy
150	~7.5	27.8%	145/150	✗ SLO broken
300	~7.0	40.4%	236/300	✗ Collapsed

5.4 Adaptive Complexity A/B

Figure 3 presents complexity trajectories and cumulative pacing violations under a 150-session ramp. Mode A (sessions) reduces violations from 21.2% to 15.6% (26% improvement) and stabilises at c=3 after crossing the second session threshold. Mode B (drift) achieves 14.8% violations but oscillates continuously between c=3 and c=5, re-raising complexity each time pacing momentarily recovers and re-breaking the SLO under sustained load. Mode A is strictly preferred for production: the marginal quality improvement of Mode B (0.8 percentage points) does not justify unpredictable quality oscillation.

6 Discussion

6.1 Scalability Ceiling and Horizontal Scale

The practical session ceiling on a single host is bounded by two independent limits: Opus CPU (~107 sessions/core) and pion's [8] per-connection goroutine overhead (~50–65 goroutines). At n=300, pion spawns ~17,800 proxy-side goroutines — a distinct scheduling and memory pressure ceiling. Because the proxy is a **share-nothing** design, vertical scaling on one reachable host is the intended scaling axis; horizontal scaling additionally requires a TURN relay for media reachability (media travels UDP directly to the pod, and private

cluster node IPs are unreachable without NAT traversal) plus session-routing infrastructure such as LiveKit or Pipecat for affinity and reconnect across nodes.

6.2 Share-Nothing Failover: Lossy by Design

Each session is **end-to-end and share-nothing**: the pion [8] WebRTC [1] peer connection, the provider WebSocket, the conversation context, and the encoder and jitter buffers all live in one process's memory and kernel file descriptors. This buys horizontal scalability and fault isolation but explicitly *not* state recovery. A realtime session has strong state affinity — no peer process can inherit another's file descriptor — so seamless L4 connection migration is impractical and is not pursued. **When a node fails, the in-flight session state is lost.**

What the design provides instead is best-effort reconstruction, not guaranteed recovery. Transcript events are externalised to a replayable Kafka [6] log keyed by `user_id` with a monotonic `seq`, and `session_id` is server-minted. On reconnect the client presents `X-Session-ID` and `X-Last-Seq`; the proxy attempts replay — first from the same node's in-memory archive, then (with `--replay-kafka`) from a bounded Kafka tail scan (300 ms budget, 100-line cap). This restores *our* session metadata and transcribed text. It does **not** restore the provider's dialogue context, which lives inside the upstream provider's server-side WebSocket connection and is unreachable to us. Recovery is therefore **partial and not stable by nature**: we can resume the transcript view, but cannot guarantee the upstream end-to-end model resumes mid-thought. We classify this honestly as L2–L3 failover (session / progress resume, best-effort), never L4 (seamless migration).

6.3 Resampling Quality

The current resampler uses linear interpolation. At our integer ratios (48 kHz ↔ 16 kHz [2], 24 kHz → 48 kHz) per-chunk boundaries align exactly and artefacts are minimal for 16 kbps narrowband voice. For non-integer ratios or higher-quality applications a polyphase FIR filter should be substituted.

7 Conclusion

We have presented RT-LLM-Proxy, a production-grade real-time audio bridge from WebRTC [1] browsers to streaming end-to-end LLM voice providers. Three engineering contributions — clock-drift-free Ticker-based pacing, zero-allocation codec buffer reuse, and proactive adaptive Opus [2] complexity control — address the principal performance challenges of real-time audio at scale. The resulting system achieves ~107 sessions/core, zero allocation overhead on the audio hot path, and 26% improvement in pacing SLO violations under adaptive complexity management.

The system follows a clear design philosophy: make the trade-offs explicit, fail open on the control plane, and never gate the real-time data plane on best-effort side concerns. Its share-nothing structure scales vertically and isolates faults, at the honest cost of *lossy failover* — session state is reconstructed best-effort from a bounded Kafka [6] replay, never seamlessly migrated, and the upstream model's dialogue context cannot be guaranteed to resume. The invariants that enforce these boundaries — Redis stays off the media path, the side-channel is drop-on-full, replay is bounded — are documented and tested, not assumed. We believe this clarity is as much a contribution as the performance numbers.

References

[1] H. Alvestrand. *Overview: Real-Time Protocols for Browser-Based Applications.* RFC 8825, IETF, January 2021.

[2] J.-M. Valin, K. Vos, and T. Terriberry. *Definition of the Opus Audio Codec.* RFC 6716, IETF, September 2012.

[3] H. Schulzrinne, S. Casner, R. Frederick, and V. Jacobson. *RTP: A Transport Protocol for Real-Time Applications.* RFC 3550, IETF, July 2003.

[4] J. Spittka, K. Vos, and J.-M. Valin. *RTP Payload Format for the Opus Speech and Audio Codec.* RFC 7587, IETF, June 2015.

[5] ITU-T. *One-Way Transmission Time.* ITU-T Recommendation G.114, International Telecommunication Union, Geneva, May 2003.

[6] J. Kreps, N. Narkhede, and J. Rao. *Kafka: A Distributed Messaging System for Log Processing.* In *Proc. 6th International Workshop on Networking Meets Databases (NetDB)*, Athens, Greece, June 2011.

[7] M. T. Nygard. *Release It! Design and Deploy Production-Ready Software*, 2nd ed. Pragmatic Bookshelf, Raleigh, NC, 2018.

[8] S. Murley et al. *pion/webrtc: A Pure Go Implementation of the WebRTC API.* GitHub, 2018. https://github.com/pion/webrtc.

[9] LiveKit Inc. *LiveKit: Open Source Realtime Communication Infrastructure.* GitHub, 2021. https://github.com/livekit/livekit.

[10] Daily.co. *Pipecat: Open Source Framework for Real-Time Conversational AI Pipelines.* GitHub, 2024. https://github.com/pipecat-ai/pipecat.

[11] R. Jesup, S. Loreto, and M. Tüxen. *WebRTC Data Channels.* RFC 8831, IETF, January 2021.

Appendix A Complete Module Architecture

Figure 5 expands Figure 1 to show all Go packages, the full offer-path module chain, data-plane internals, optional backends, and the fault-tolerance policy applied at each component.

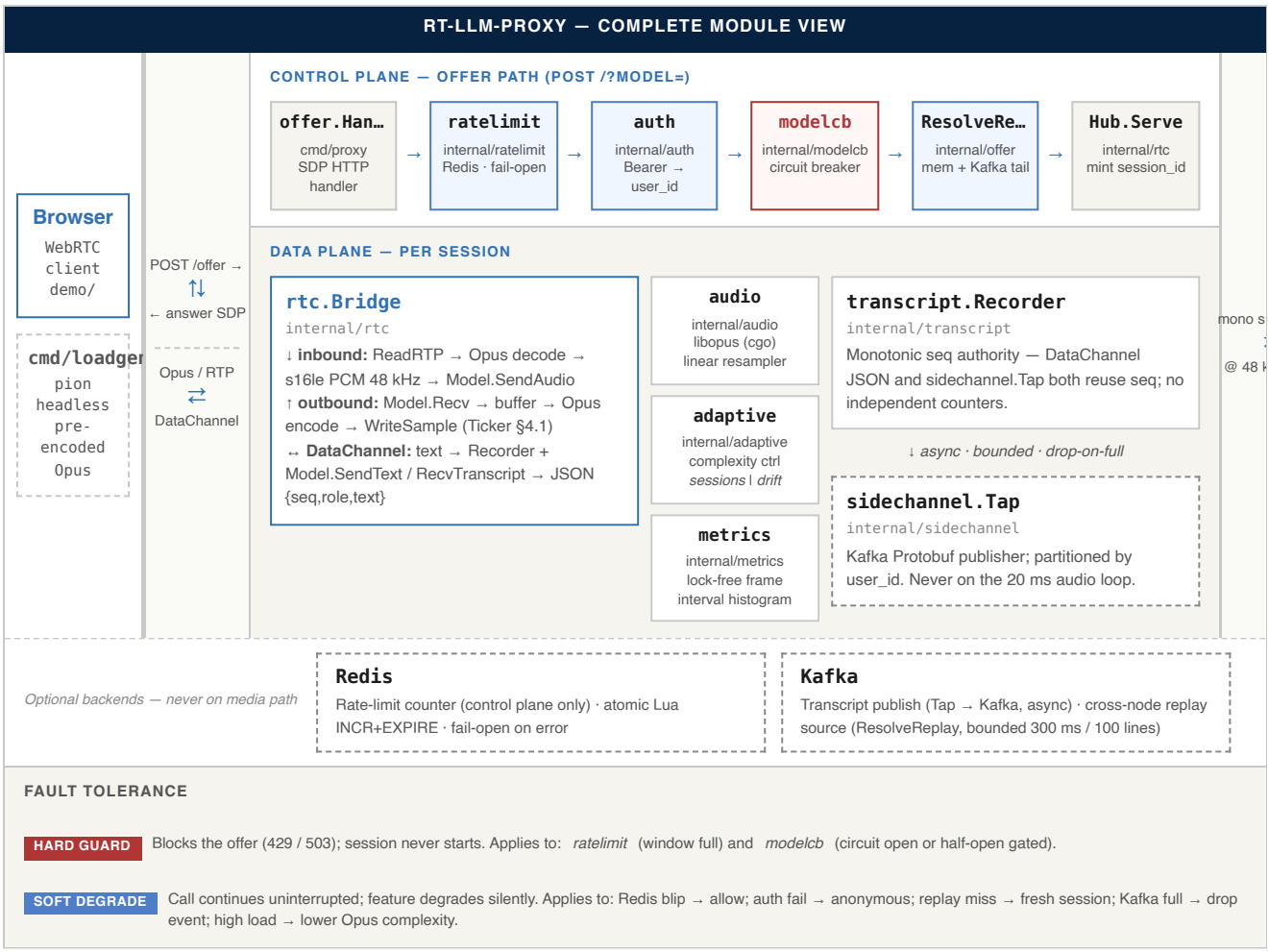

Figure 5. Complete module architecture. Solid borders = hot path; dashed borders = optional or best-effort paths. Blue-tinted boxes = soft-degrade on failure; red-tinted = hard guard. Arrows between the browser and the proxy, and between the proxy and the LLM provider, carry the audio and DataChannel traffic on the data plane, and SDP on the control plane.